

BACKEND TASKS · REVISION 2

Admin & Super Admin Operations Dashboard

An agency-wide operations view for both administrator roles, and the client administration actions behind it.

REVISION

2 — 25 August 2026

SCOPE

34 tasks · 18 epics

BACKEND

NestJS 11 · PostgreSQL

WHAT CHANGED IN THIS REVISION

The operations dashboard is no longer Super-Admin-only. **Admin** (AGENCY_ADMIN) now gets the dashboard plus client administration — add a client, edit a client name, assign a client to users. Two capabilities stay **Super Admin only**: assigning admins, and deleting a client from the database.

Epic 0 and **Epic 17** are new; **BE-01**, **BE-02**, **BE-29** and **BE-30** were rewritten. Everything else is unchanged from revision 1.

0 Role rules NEW

Read this before Epic 1. Every task below inherits these rules; where a task repeats them it is for emphasis, not as a separate decision.

VOCABULARY

This document	Code	Notes
Super Admin	<code>PlatformRole.SUPER_ADMIN</code>	Full platform control
Admin	<code>PlatformRole.AGENCY_ADMIN</code>	Agency operations — no destructive or role-granting powers
Employee	<code>PlatformRole.USER</code>	Works on the clients they are assigned to
Client user	<code>USER + CLIENT_OWNER / CLIENT_REVIEWER</code>	External. Never an administrator

"Client" in the interface is `company` in the API. A client is assigned to a user through a `company_membership`.

PERMISSION MATRIX

Capability	Super Admin	Admin	Employee	Client user
Operations dashboard — all of <code>/api/admin/dashboard/*</code>	yes	yes	403	403
Add client	yes	yes	403	403
Edit client name	yes	yes	403	403
Assign client to users — create, re-role, remove membership	yes	yes	403	403
Assign admins — grant or revoke <code>AGENCY_ADMIN</code>	yes	403	403	403
Delete client from the database	yes	403	403	403
System health — <code>/api/admin/system/health</code>	yes	403	403	403

RULES

R1 — Same dashboard for both admin roles. Admin and Super Admin receive identical dashboard payloads. Admin is *not* narrowed to the clients they hold a membership on: the dashboard is an agency-wide operations view for both roles.

R2 — Admin runs day-to-day client administration. Creating a client, renaming it and staffing it with employees are Admin-level actions.

R3 — Only Super Admin changes a platform role. Granting or revoking `AGENCY_ADMIN` (and `SUPER_ADMIN`) is Super Admin only. An Admin must not be able to promote themselves or anyone else.

R4 — Only Super Admin deletes a client. A hard delete removes the client and its dependent records from the database. Admin may deactivate or archive a client through its status, but never delete it.

R5 — Everyone else is refused. `USER` and client users receive `403` on every route under `/api/admin/*`, with no data leaked in the error body.

R6 — Platform health stays with Super Admin. `/api/admin/system/health` exposes infrastructure state, not agency operations, so it remains Super Admin only. *Assumption — confirm with the Product Owner if Admin should also see it.*

IMPLEMENTATION NOTE

The role machinery already exists in the codebase; use it rather than inventing a new mechanism.

```
// Dashboard endpoints – both admin roles
@UseGuards(JwtAuthGuard, PlatformRolesGuard)
@PlatformRoles(PlatformRole.SUPER_ADMIN, PlatformRole.AGENCY_ADMIN)

// Assign admins, delete client, system health – Super Admin only
@UseGuards(JwtAuthGuard, PlatformRolesGuard)
@PlatformRoles(PlatformRole.SUPER_ADMIN)
```

TWO EXISTING GAPS THIS MATRIX EXPOSES

1. PATCH `/api/companies/:companyId` is protected by `CompanyAccessGuard` only, so any member of the client can rename it. It must require an admin platform role — **BE-31**.
2. PATCH `/api/users/:userId` accepts `platformRole` while the whole `UsersController` admits both admin roles, so an Admin can currently promote anyone. Role changes must be split out and restricted to Super Admin — **BE-33**.

1 Operations dashboard foundation

BE-01 Create the Admin dashboard module REVISED

- Create `AdminDashboardModule`, `AdminDashboardController`, `AdminDashboardService`
- Restrict all dashboard endpoints to `SUPER_ADMIN` and `AGENCY_ADMIN`
- Do not require a `companyId`
- Allow dashboard queries to aggregate data across all clients
- Both admin roles receive the same data — do not scope Admin to their memberships (R1)
- Reuse existing repositories/services where possible

SUGGESTED BASE ROUTE

```
/api/admin/dashboard
```

ACCEPTANCE CRITERIA

- `SUPER_ADMIN` and `AGENCY_ADMIN` can retrieve cross-client information
- `USER` and client users receive `403`
- Dashboard endpoints do not depend on selecting a client first
- An Admin with no company memberships still sees every client's data

BE-02 Add common dashboard filters REVISED

Support reusable query parameters: `from`, `to`, `clientId`, `employeeId`, `status`, `priority`.

Default behaviour: `from/to` → today · `clientId` → all clients · `employeeId` → all employees.

ACCEPTANCE CRITERIA

- The same filtering rules are reused across dashboard endpoints
- Invalid dates/IDs return `400`
- Both Admin and Super Admin can filter one client without entering the client workspace
- `clientId` filtering is not restricted by the caller's memberships

2 Agency overview

BE-03 Create dashboard overview endpoint

```
GET /api/admin/dashboard/overview
```

Return active clients, active employees, work due today, overdue work, blocked tasks, posts waiting for client approval, leads requiring follow-up today, overdue lead follow-ups, posts published today.

```
{
  "activeClients": 18,
  "activeEmployees": 24,
  "dueToday": 42,
  "overdue": 11,
  "blockedTasks": 4,
  "waitingClientApproval": 8,
  "leadFollowUpsToday": 13,
  "overdueLeadFollowUps": 5,
  "publishedToday": 9
}
```

ACCEPTANCE CRITERIA

- Values are calculated across all clients
- Filters work correctly
- No duplicate work items are counted

3 Attention / problems feed

BE-04 Build the "Needs Attention" aggregation endpoint

```
GET /api/admin/dashboard/attention
```

Return operational exceptions: overdue tasks, blocked tasks, overdue publishing tasks, client approvals waiting too long, changes-requested posts waiting for action, lead follow-ups overdue, campaigns ending soon with incomplete work, unassigned urgent/high-priority tasks.

```
{
  "type": "TASK_OVERDUE",
  "severity": "CRITICAL",
  "client": {},
  "entityId": "...",
  "title": "...",
  "owner": {},
  "dueAt": "...",
  "waitingSince": "...",
  "ageMinutes": 1280
}
```

Severity — implement `INFO`, `WARNING`, `CRITICAL`.

ACCEPTANCE CRITERIA

- Results ordered by severity and urgency; critical issues appear first
- Every record contains enough information for frontend drill-down
- Endpoint supports pagination

4 Content process monitoring

BE-05 Build content pipeline metrics

```
GET /api/admin/dashboard/content
```

Return counts for `DRAFT`, `IN_INTERNAL_REVIEW`, `READY_FOR_CLIENT`, `CHANGES_REQUESTED`, `APPROVED`, `SCHEDULED`, `PUBLISHED`, `CANCELED`.

Also return published today, scheduled today, awaiting client action, changes requested, overdue scheduled posts, average approval waiting time.

BE-06 Detect scheduled posts requiring manual publishing

The platform stores scheduled posts but does not publish them automatically:

- Find posts with `status = SCHEDULED`
- Compare `scheduledAt <= now`
- Exclude already published posts
- Return them as `PUBLISHING_DUE` and include them in `/attention`

ACCEPTANCE CRITERIA

- A scheduled post becomes visible as due when its scheduled time passes
- No automatic social publishing is required in this task

5 Client approval monitoring

BE-07 Create approvals dashboard endpoint

```
GET /api/admin/dashboard/approvals
```

Return total waiting for client, waiting by client, oldest waiting post, average waiting duration, changes requested count, approved today, rejected/canceled today.

```
{
  "waiting": 12,
  "overThreshold": 3,
  "clients": [
    {
      "clientId": "...",
      "clientName": "Taxero",
      "waiting": 4,
      "oldestWaitingHours": 49,
      "changesRequested": 1
    }
  ]
}
```

BE-08 Add approval aging calculation

For each post waiting for approval: determine when it entered `READY_FOR_CLIENT`, calculate the current waiting duration, provide `waitingHours`, and provide a derived SLA state — `NORMAL`, `WARNING`, `CRITICAL`.

Keep thresholds configurable rather than hard-coded where possible.

6 Task operations

BE-09 Create tasks dashboard metrics

```
GET /api/admin/dashboard/tasks
```

Return total open tasks, `TODO`, `IN_PROGRESS`, `IN_REVIEW`, `BLOCKED`, due today, overdue, urgent, high priority, unassigned, completed today.

BE-10 Create employee workload aggregation

```
GET /api/admin/dashboard/team-workload
```

For each employee return active clients count, total active tasks, due today, overdue, blocked, in review, urgent/high-priority work.

```
{
  "employeeId": "...",
  "name": "Ahmad",
  "clients": 3,
  "openTasks": 12,
  "dueToday": 4,
  "overdue": 4,
  "blocked": 2,
  "urgent": 1
}
```

IMPORTANT

Do not calculate an arbitrary workload percentage yet. Return objective workload metrics first.

7 Lead operations

BE-11 Create leads dashboard endpoint

```
GET /api/admin/dashboard/leads
```

Return counts for NEW, CONTACTED, INTERESTED, WAITING_DECISION, WON, LOST, FOLLOW_UP_LATER.

Also return new leads today, follow-ups today, overdue follow-ups, won this month, lost this month, conversion rate.

BE-12 Create overdue lead-follow-up detection

```
followUpDate < now
AND lead is not WON
AND lead is not LOST
```

Return lead, client, assigned employee, follow-up date, number of hours/days overdue.

8 Campaign monitoring

BE-13 Create campaign dashboard endpoint

```
GET /api/admin/dashboard/campaigns
```

Return active, draft, paused and completed campaigns, campaigns ending soon, campaigns with overdue tasks.

For each active campaign: client, objective, start/end date, total linked posts, published posts, total linked tasks, completed tasks, linked leads, won leads.

BE-14 Create campaign progress aggregation

```
{
  "tasks": { "total": 16, "completed": 12 },
  "posts": { "total": 10, "published": 8 },
  "leads": { "total": 43, "won": 7 }
}
```

Do not generate one arbitrary campaign percentage unless product rules for the calculation are defined.

9 Content plan monitoring

BE-15 Create monthly content plan overview

```
GET /api/admin/dashboard/content-plans
```

Return `DRAFT`, `INTERNAL_REVIEW`, `CLIENT_REVIEW`, `APPROVED`, `ARCHIVED`. Group by client.

BE-16 Detect clients missing current-month content plans

Create a derived dashboard state `MISSING`. This must **not** be added to the database content-plan enum.

```
active client + no content plan for selected month = MISSING
```

Return client, account manager if available, selected month/year.

10 Client health data

BE-17 Create client operations summary

```
GET /api/admin/dashboard/clients
```

For every active client return open tasks, overdue tasks, blocked tasks, content awaiting approval, changes requested, scheduled posts, published posts, open leads, overdue lead follow-ups, active campaigns, current content-plan status.

```
{
  "clientId": "...",
  "clientName": "Taxero",
  "tasks": { "open": 8, "overdue": 1, "blocked": 0 },
  "content": { "waitingApproval": 2, "changesRequested": 1 },
  "leads": { "open": 13, "overdueFollowUps": 1 }
}
```

BE-18 Prepare client health score inputs

Do **not** hard-code the final score yet. Expose the underlying metrics the frontend/product layer needs: overdue task ratio, blocked task count, client approval age, overdue lead follow-ups, current content-plan status, campaign issues.

The Product Owner should define the final scoring formula separately.

11 Automations monitoring

BE-19 Create automation dashboard summary

```
GET /api/admin/dashboard/automations
```

Return runs today, successful runs, failed runs, active rules, inactive rules, last failed runs.

For failed runs return automation rule, client, trigger, action, failure timestamp, error where available.

12 Recent agency activity

BE-20 Design global audit/event model

The platform has some entity-specific logs but no complete system-wide audit trail. Create a reusable global activity entity.

```
id · actorUserId · clientId · entityType · entityId · action · metadata · createdAt
```

Potential actions: TASK_CREATED, TASK_STATUS_CHANGED, POST_CREATED, POST_SUBMITTED, POST_APPROVED, POST_CHANGES_REQUESTED, POST_PUBLISHED, LEAD_CREATED, LEAD_STATUS_CHANGED, CAMPAIGN_CREATED, MEMBER_ADDED, AUTOMATION_EXECUTED.

Add the administrative actions from Epic 17 as well: CLIENT_CREATED, CLIENT_RENAMED, CLIENT_DELETED, MEMBER_REMOVED, PLATFORM_ROLE_CHANGED.

BE-21 Write important platform actions into the global activity log

Integrate audit recording into major workflows: posts, approvals, tasks, leads, campaigns, users/memberships, automations, and client administration (Epic 17).

Prefer centralized event/subscriber logic rather than duplicating audit-writing code everywhere.

BE-22 Create recent activity endpoint

```
GET /api/admin/dashboard/activity
```

Support pagination, client filter, user filter, entity-type filter and date range. Return newest first.

13 Platform health

BE-23 Create Super Admin system-health endpoint

```
GET /api/admin/system/health
```

Super Admin only — the one route in the admin area that Admin does not reach (R6).

Check API, PostgreSQL, object storage (MinIO or S3), Redis when actually used, scheduler when implemented, AI provider configuration, email/SMTP configuration.

```
{
  "api": "UP",
  "database": "UP",
  "storage": "UP",
  "redis": "NOT_IN_USE",
  "scheduler": "NOT_IMPLEMENTED",
  "email": "NOT_CONFIGURED",
  "ai": "MOCK"
}
```

Do not report services as healthy simply because environment variables exist. Verify connectivity when applicable.

14 Dashboard API performance

BE-24 Optimize dashboard queries

- Avoid N+1 client queries
- Aggregate with database queries where appropriate
- Use `COUNT`, `GROUP BY`, conditional aggregation
- Add required indexes after reviewing query plans
- Avoid loading complete entities when only counts are required

Target: the main dashboard request ideally under `500 ms` with realistic agency data.

BE-25 Decide dashboard API composition

Option A — `GET /api/admin/dashboard` returns all major sections. **Option B** — separate `/overview`, `/attention`, `/content`, `/tasks` endpoints.

Recommended: several domain endpoints plus one lightweight `/overview` endpoint. Widgets then refresh independently, and one expensive query cannot block the entire dashboard.

15 Pagination and consistency

BE-26 Add pagination to dashboard detail endpoints

Required for attention items, activity, tasks drill-down, approval records, leads, and client summaries where necessary. Standardize `?page=1&limit=20`.

```
{
  "items": [],
  "pagination": { "page": 1, "limit": 20, "total": 156, "totalPages": 8 }
}
```

BE-27 Standardize dashboard time calculations

Create shared helpers for start/end of day, selected timezone, due today, overdue, waiting duration, current month, published today.

Do not scatter `new Date()` comparison logic throughout different services.

16 Testing

BE-28 Add unit tests for dashboard aggregation

Test at minimum: due-today calculations, overdue tasks, blocked tasks, client approval aging, scheduled publishing due, lead follow-up overdue, missing monthly content plan, and admin filtering by `clientId`.

BE-29 Add authorization tests REVISED

EXPECTED RESULT PER ROLE

Role	Dashboard	Add client · edit name · assign to users	Assign admins	Delete client	System health
SUPER_ADMIN	allowed	allowed	allowed	allowed	allowed
AGENCY_ADMIN	allowed	allowed	403	403	403
USER	403	403	403	403	403
Client user	403	403	403	403	403

Also verify:

- An `AGENCY_ADMIN` cannot promote themselves or anyone else to `AGENCY_ADMIN` / `SUPER_ADMIN` through any route, including `PATCH /api/users/:userId`
- An `AGENCY_ADMIN` calling `DELETE /api/companies/:companyId` gets `403` and the client row still exists afterwards
- A `403` response body leaks no client or user data

BE-30 Add integration tests for cross-client isolation REVISED

Create at least two clients with independent data. Verify:

- A Super Admin sees both
- An Admin sees both, including a client they hold no membership on
- The `clientId` filter returns only the selected client
- No duplicate cross-client records
- A normal user and a client user cannot access the admin endpoints

17 Client administration NEW

The dashboard is read-only; these are the write actions behind it. All four tasks live in the existing `companies` / `users` modules — no new module is required.

BE-31 Add client and edit client name ADMIN + SUPER ADMIN

```
POST /api/companies
PATCH /api/companies/:companyId
```

- Guard both with `PlatformRolesGuard` + `@PlatformRoles(SUPER_ADMIN, AGENCY_ADMIN)`
- `POST /api/companies` is already correct — leave the rule as it is
- **Fix required:** `PATCH /api/companies/:companyId` currently runs `CompanyAccessGuard` only, so any member of that client can rename it. Add the platform-role requirement
- Renaming a client must not change its `id`, memberships or any linked records
- Record `CLIENT_CREATED` / `CLIENT_RENAMED` in the global activity log (BE-21)

ACCEPTANCE CRITERIA

- Admin and Super Admin can create a client and rename it
- An employee with an active membership on the client receives `403` when renaming it
- The name is validated (2–160 characters) and duplicates are handled per product rules

BE-32 Assign a client to users ADMIN + SUPER ADMIN

```
POST /api/companies/:companyId/members
PATCH /api/companies/:companyId/members/:membershipId
DELETE /api/companies/:companyId/members/:membershipId
```

- Keep the platform-role requirement already present on `POST` and apply the same rule to `PATCH` and `DELETE` — staffing decisions are administrative, not per-client
- Assigning a user who is already an active member must not create a duplicate membership
- Record `MEMBER_ADDED` / `MEMBER_REMOVED` in the global activity log

ACCEPTANCE CRITERIA

- Admin and Super Admin can assign, re-role and remove a user on any client
- An Account Manager without an admin platform role can no longer staff a client
- Assignments are visible immediately in `/team-workload` and `/clients`

BE-33 Assign admins SUPER ADMIN ONLY

```
PATCH /api/users/:userId/platform-role
```

- New dedicated route, `@PlatformRoles(SUPER_ADMIN)` only
- **Fix required:** remove `platformRole` from `UpdateUserDto`, so `PATCH /api/users/:userId` — open to both admin roles — can no longer change a role. `CreateUserDto.platformRole` must be rejected with `403` unless the caller is a Super Admin
- Guard rails: a Super Admin cannot demote themselves, and the last active `SUPER_ADMIN` cannot be demoted or deactivated
- Record `PLATFORM_ROLE_CHANGED` — previous role, new role, actor — in the activity log
- Existing sessions must pick up the new role: revoke refresh tokens or re-issue on next refresh, since the role is carried in the JWT

ACCEPTANCE CRITERIA

- Super Admin can grant and revoke `AGENCY_ADMIN`
- Admin receives `403` on this route, including when the target is themselves
- The last remaining Super Admin cannot lose the role
- A demoted Admin loses dashboard access without needing a manual database change

BE-34 Delete a client from the database SUPER ADMIN ONLY

```
DELETE /api/companies/:companyId
```

- New route, `@PlatformRoles(SUPER_ADMIN)` only
- This is a **hard delete** — the client row is removed from PostgreSQL, not archived. Deactivating a client stays a status change via `PATCH` and remains available to Admin
- Define and document the cascade before implementing: memberships, tasks, posts, content plans, approvals, leads, campaigns, automations, files, notifications and audit rows all reference the client. Decide per table whether it cascades, is nulled, or blocks the delete
- Uploaded files in object storage must be removed or deliberately orphaned — state which
- Run the whole delete in a single transaction; on any failure nothing is removed
- Require an explicit confirmation — for example `?confirm=<client name>` — and return `409` if it does not match
- Write `CLIENT_DELETED` to the activity log **before** the delete commits, with a snapshot of the client name and the deleted row counts, so the record survives the client's removal

ACCEPTANCE CRITERIA

- Super Admin can delete a client and it is gone from the database
- Admin, employees and client users receive `403` and the client still exists
- No orphaned rows remain in any client-scoped table afterwards
- The dashboard endpoints do not error for other clients after a delete
- The activity log still shows who deleted which client and when

Revision 2 · 25 August 2026 · Solutions Platform backend. Supersedes “Backend Tasks — Super Admin Operations Dashboard” (revision 1). Source: `api/docs/backend-tasks-admin-dashboard.html`; text version: `api/docs/BACKEND-TASKS-ADMIN-DASHBOARD.md`.